Introduction to the Theory of Computation

Lecture 16. Complexity Class P

Neo (Yuan) Li

Oct 2026

Fudan University

yuan_li@fudan.edu.cn

Definition of DTIME

Definition 1
Let $T : \mathbb{N} \to \mathbb{N}$. A language $L \subseteq \{0,1\}^*$ is in $\mathrm{DTIME}(T(n))$ if some multi-tape TM decides L in time $O(T(n))$.

Space Used by a Turing Machine

Definition 2

Let M be a multi-tape TM with a read-only input tape. The *space used by* M on input x is the number of distinct work-tape cells visited during the computation.

M runs in space $S(n)$ if it uses at most $S(n)$ cells on every input of length n.

Definition of DSPACE

Definition 3

Let $S : \mathbb{N} \to \mathbb{N}$. A language $L \subseteq \{0,1\}^*$ is in $\mathrm{DSPACE}(S(n))$ if some multi-tape TM decides L in space $O(S(n))$.

Time Complexity Classes: P and EXP

Definition 4

$$\mathrm{P} = \bigcup_{k \geq 1} \mathrm{DTIME}(n^k)$$

Definition 5

$$\mathrm{EXP} = \bigcup_{k \geq 1} \mathrm{DTIME}\left(2^{n^k}\right)$$

P: decidable in **polynomial time**.

EXP: decidable in **exponential time**.

Space Complexity Classes: L, PSPACE

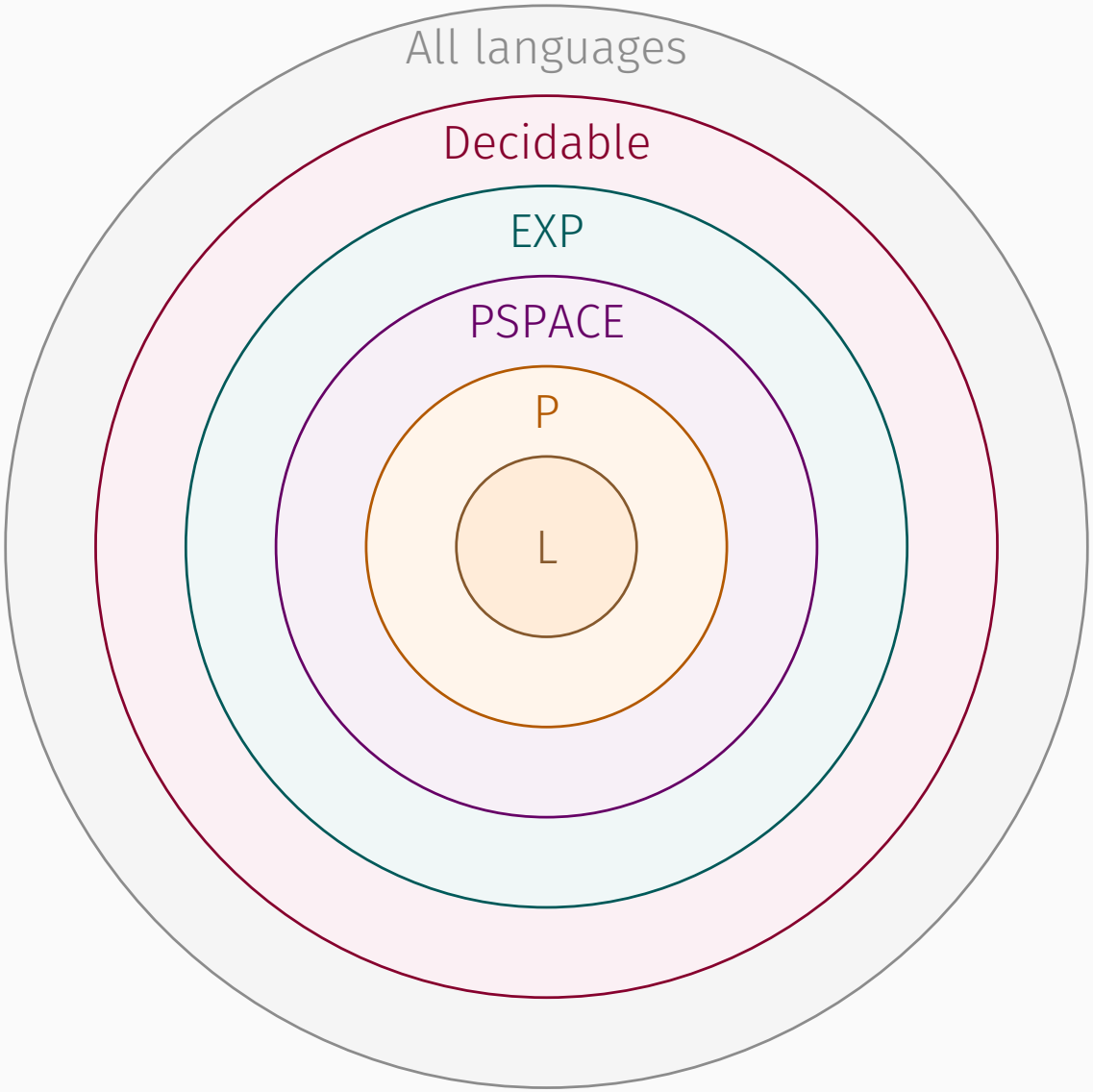
Definition 6

$$\mathrm{L} = \mathrm{DSPACE}(\log n) \qquad \text{(also called LOGSPACE)}$$

Definition 7

$$\mathrm{PSPACE} = \bigcup_{k \geq 1} \mathrm{DSPACE}(n^k)$$

The Landscape



The Landscape

- Every inclusion in

$$\mathsf{L} \subseteq \mathsf{P} \subseteq \mathsf{PSPACE} \subseteq \mathsf{EXP}$$

  is open.
- But the **Hierarchy Theorems** give

$$\mathsf{L} \subsetneq \mathsf{PSPACE}, \qquad \mathsf{P} \subsetneq \mathsf{EXP}.$$

Remark 1: Model Independence

- Single-tape TMs, multi-tape TMs, and RAM machines are **polynomially equivalent**: each can simulate the others with polynomial overhead.
- Hence the definition of P does not depend on the choice of model.

Remark 2: Why Ignore the Exponent?

- P is an idealization: n^{100} is not practical. Empirically, though, natural problems in P tend to have small exponents, which keep improving.
- Exponential time typically comes from brute-force search.

Remark 3: Closure under Composition

- Polynomials are closed under $+$, $\times$, and composition: $f(n) + g(n)$, $f(n) \cdot g(n)$, $f(g(n))$ are all polynomials.
- Hence combining polynomial-time subroutines yields a polynomial-time program.

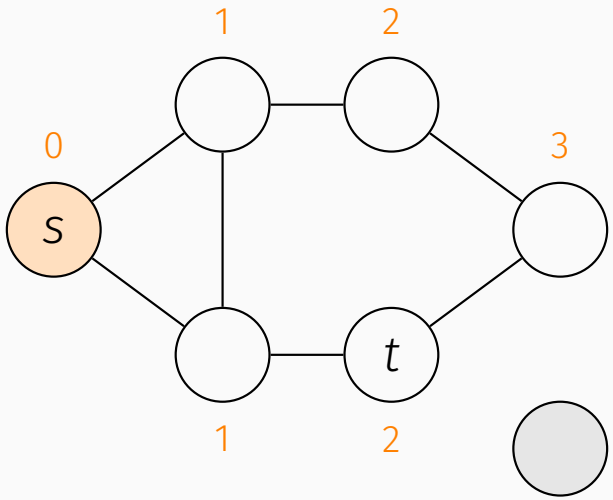
“Efficient” = Polynomial Time

- **Algorithms**: efficient = polynomial time.
- **Cryptography**: secure = no polynomial-time adversary can break it.
- **Learning**: a concept class is **efficiently learnable** with polynomially many samples and polynomial time.

Graph Connectivity

$\mathrm{PATH} = \{\langle G, s, t\rangle : G \text{ has a path from } s \text{ to } t\}$.

- **BFS** or **DFS** from s; accept iff t is reached.
- Linear time in the input size, for either representation:
  - adjacency list: $O(n + m)$
  - adjacency matrix: $O(n^2)$



Numbers: BFS layers. Gray: unreachable.

String Matching

Given pattern p ($|p| = m$) and text T ($|T| = n$): does p occur in T?

- Naive: $O(nm)$.
- **KMP** (1977): $O(n + m)$, linear time.
- In practice: **Boyer–Moore**-style algorithms, linear in the worst case and often faster on average.

Union-Find: Almost Linear

Maintain disjoint sets under two operations:

- $\mathrm{Union}(x, y)$: merge the sets of x and y.
- $\mathrm{Same}(x, y)$: are x and y in the same set?

m operations on n elements: $O(m\,\alpha(n))$ time.

- α: inverse Ackermann. $\alpha(n) \leq 4$ for any practical n.

Shortest Paths

Given a weighted graph G and vertices s, t:
find a shortest path from s to t.

Weights	Algorithm	Time
unweighted	BFS	$O(n + m)$
non-negative	Dijkstra	$O(m + n \log n)$
negative allowed	Bellman–Ford	$O(nm)$

n: number of vertices; m: number of edges.

Minimum Spanning Tree

Given a weighted connected graph G:
find a spanning tree of minimum total weight.

Algorithm	Idea	Time
Kruskal 1956	sort edges, Union-Find	$O(m \log n)$
Prim 1957	one tree, like Dijkstra	$O(m + n \log n)$
Karger–Klein–Tarjan 1995	random sampling	$O(m)$ expected
Chazelle 2000	soft heap	$O(m\,\alpha(m, n))$
Pettie–Ramachandran 2002	time unknown	$\Omega(m)$ to $O(m\,\alpha)$

n: number of vertices; m: number of edges.

Deterministic linear time: open.

Maximum Flow

Given a graph G with edge capacities and vertices s, t: find a maximum flow from s to t.

Algorithm	Idea	Time
Ford–Fulkerson 1956	augmenting paths	$O(m \cdot f)$, not poly
Edmonds–Karp 1972	shortest augmenting path	$O(nm^2)$
Dinic 1970	blocking flows	$O(n^2 m)$
Goldberg–Tarjan 1988	push–relabel	$O(n^2\sqrt{m})$
Orlin 2013	strongly polynomial	$O(nm)$
Chen et al. 2022	continuous optimization	$m^{1+o(1)}$

n: number of vertices; m: number of edges; f: value of the max flow.

Linear Programming

$$\max\ c^{\mathsf{T}}x \quad \text{subject to} \quad Ax \leq b,\ x \geq 0$$

Algorithm	Idea	Time
Simplex (Dantzig 1947)	walk on vertices	exponential
Ellipsoid (Khachiyan 1979)	shrinking ellipsoids	$O(n^4 L)$
Interior point (Karmarkar 1984)	through the interior	$O(n^{3.5} L)$
Vaidya 1989	faster linear algebra	$O(n^{2.5} L)$
Cohen–Lee–Song 2019	$\approx$ matrix multiplication	$\tilde{O}(n^{\omega})$

n: number of variables; L: bit-length of the input; ω: matrix multiplication exponent.

Matrix Multiplication

Given $n \times n$ matrices A, B, compute AB.

Algorithm	Idea	Time
Schoolbook	definition	$O(n^3)$
Strassen 1969	7 products for 2×2	$O(n^{2.81})$
Coppersmith–Winograd 1990	laser method	$O(n^{2.376})$
Alman et al. 2025	refined laser method	$O(n^{2.3714})$
DeepMind 2026	laser + AlphaEvolve	$O(n^{2.3712})$
OpenAI 2026	AI model	$O(n^{2.25})$*

*preprint, Oct 2026, not yet peer reviewed

- Is $\omega = 2$? Open.
- Checking $AB = C$: $O(n^2)$ randomized (Freivalds 1977).

Primality Testing

$\mathrm{PRIMES} = \{\langle N \rangle : N \text{ is prime}\}$, input length $n = \log_2 N$.

Algorithm	Idea	Time
Trial division	try all $d \le \sqrt{N}$	$2^{n/2}$, exponential
Miller–Rabin 1976/80	randomized	$\tilde{O}(n^2)$
AKS 2002	deterministic	$\tilde{O}(n^6)$

- Contrast: FACTORING: no polynomial-time algorithm known.

Integer Multiplication and FFT

Multiply two n-bit integers.

Algorithm	Idea	Time
Schoolbook	digit by digit	$O(n^2)$
Karatsuba 1960	3 products, not 4	$O(n^{1.585})$
Schönhage–Strassen 1971	FFT	$O(n \log n \log\log n)$
Harvey–van der Hoeven 2019	refined FFT	$O(n \log n)$
OpenAI 2026	AI model	$O(n(\log n)^{1-\kappa})$*

$\kappa = 2^{-182}$. *preprint, Sept 2026, not yet peer reviewed

- **FFT** (Cooley–Tukey 1965): Fourier transform in $O(n \log n)$.

Edit Distance

$\mathrm{ed}(x, y)$: the minimum number of **edits** to turn x into y.
Edit $=$ insert, delete, or substitute one character.

$$\texttt{kitten} \xrightarrow{\text{substitute}} \texttt{sitten} \xrightarrow{\text{substitute}} \texttt{sittin} \xrightarrow{\text{insert}} \texttt{sitting}$$

$$\mathrm{ed}(\texttt{kitten}, \texttt{sitting}) = 3$$

Algorithm	Idea	Time
Wagner–Fischer 1974	dynamic programming	$O(n^2)$
Masek–Paterson 1980	"Four Russians"	$O(n^2 / \log^2 n)$

Polar Codes: Encoding and Decoding

Block length $N = 2^n$. Encode: $x = u\,F^{\otimes n}$, $\quad F = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$.

Task	Algorithm	Time
Encoding	butterfly, like FFT	$O(N \log N)$
Decoding	successive cancellation (Arikan 2009)	$O(N \log N)$
Decoding	SC list, list size L (Tal–Vardy 2015)	$O(L\,N \log N)$
Construction	choose good bit-channels	$\mathrm{poly}(N)$

- Achieves capacity; gap ε needs only $N = \mathrm{poly}(1/\varepsilon)$ (Guruswami–Xia 2013).
- In 5G: SC list decoding + CRC.

LDPC Codes: Encoding and Decoding

Codewords: $\{x : Hx = 0\}$, H a **sparse** parity-check matrix.

Task	Algorithm	Time
Encoding	generator matrix (dense)	$O(N^2)$
Encoding	Richardson–Urbanke 2001; 5G structure	$\approx O(N)$
Decoding	belief propagation (Gallager 1962)	$O(N)$ per round
Decoding	min-sum (in hardware)	$O(N)$ per round

- Within a fraction of a dB of capacity; spatially coupled LDPC provably achieves capacity (Kudekar–Richardson–Urbanke 2013).
- In Wi-Fi, 5G data channel, SSDs, satellite TV.

Reed–Solomon Codes: Encoding and Decoding

Message: polynomial f of degree $< k$ over $\mathbb{F}_q$.
Codeword: $\big(f(\alpha_1), \ldots, f(\alpha_n)\big)$.

Task	Algorithm	Time
Encoding	evaluate f at n points	$O(n^2)$; $\tilde{O}(n)$ via FFT
Decoding	Berlekamp–Massey 1968	$O(n^2)$; $\tilde{O}(n)$ fast
List decoding	Guruswami–Sudan 1999	$\mathrm{poly}(n)$

- Corrects up to $\frac{n-k}{2}$ errors: optimal (Singleton bound).
- List decoding: up to $n - \sqrt{nk}$ errors.
- In QR codes, CDs, Ethernet, deep-space probes.

Text Compression

Lossless: every bit must come back.

Algorithm	Idea	Time
Huffman 1952	frequent symbols, short codes	$O(n)$
Arithmetic coding 1976	whole message $\to$ one interval	$O(n)$
Lempel–Ziv 1977/78	repeat $\to$ pointer to earlier copy	$O(n)$
Burrows–Wheeler 1994	sort rotations, group contexts	$O(n)$
ANS (Duda) 2009	arithmetic ratio at Huffman speed	$O(n)$

- Limit: entropy. English $\approx$ 1 bit/char (Shannon 1951), vs. 8 in ASCII.

The Most-Used Algorithms in Your Day

	Algorithm	Where	Ops/day
1	Matrix multiplication	AI chat, feeds, voice input, camera	10^{12}–10^{15}
2	Error-correcting decoding	Wi-Fi/5G, flash, DRAM	10^{11}–10^{12}
3	Video/image codecs	Video, photos	10^{11}
4	Graphics rendering	Every screen refresh	10^{11}–10^{12}
5	Symmetric crypto, hashing	HTTPS, disk encryption	10^{10}
6	Public-key crypto	TLS handshakes	10^{9}–10^{10}
7	Lossless compression	Web pages, app data	10^{9}–10^{10}
8	Hash tables, sorting, B-trees	Inside all software	10^{9}
9	Routing, congestion control	Every packet	10^{8}–10^{9}
10	Search, shortest paths	Search, maps	10^{2} queries